AW CRM | Complete Project Analysis for Claude AI

Repository-based audit | 20 October 2026 | PKR / Asia-Karachi business context

Scope: /Users/wahababahmad694gmail.com/Documents/Web/aw-crm-web and adjacent /Users/wahababahmad694gmail.com/Documents/Web/aw_crm. The workspace path supplied with the request omitted /Web/ and does not exist. No backend checkout, ORM schema, migrations, database, or authenticated API session was available. This report does not contain private records or credentials.

1. Executive summary and evidence rules

AW CRM is a Next.js web client for sales, customers, catalog, stock, orders, collections, targets, expenses, reporting, follow-up, team and settings. A separate Flutter client covers a narrower set of workflows. The clients call an external versioned API. NestJS and PostgreSQL are intended architecture but cannot be independently confirmed from available source. The web AI route is unavailable; Flutter has a chat client call whose server behavior is unverified. Printing is not implemented in inspected client source.

Evidence labels: VERIFIED SOURCE means implementation exists in inspected code. HISTORICAL QA means a repository note reports live checks on 19 October 2026, but this audit did not repeat them. CODE REVIEW FINDING means a concrete source limitation. NOT VERIFIED means backend or live behavior is unavailable. RECOMMENDATION means proposed future work. Presence of a UI form or endpoint string never proves server behavior.

Highest priority: obtain backend/schema/OpenAPI and a safe QA environment; verify financial and tenant invariants; finish order/payment/refund lifecycle QA; replace manual UUID entry; then build server-issued invoices and receipts. Avoid a rewrite.

2. Verified stack and architecture

Layer	Verified source finding	Limit
Web	Next.js 16.4.2, React 19.3, TypeScript 5, App Router, npm; Tailwind 4, Base UI, React Hook Form, Zod, Recharts, Sonner. package.json	No browser visual pass in this audit
Web API	Server-side fetch, Server Actions, HTTP-only access/refresh cookies, X-Organization-Id header, no-store reads. lib/api.ts; app/login/session.ts	Server authorization unverified
Mobile	Flutter/Dart, Dio, GoRouter, Provider, GetIt, flutter_secure_storage. aw_crm/pubspec.yaml; lib/app/app.dart	Separate checkout; limited source review
Backend/database	External /api/v1 API URL configured. NestJS/PostgreSQL described in brief and docs.	No backend source, DB schema, migrations or live access; technology unverified
AI/external	Flutter POST /ai/chat; web unavailable page.	Provider, model, tools, cost, safeguards unverified

Architecture diagram - verified edges and unknowns

Web browser -> Next.js Server Components / Server Actions -> external /api/v1 service -> [database unknown]. Flutter app -> Dio client -> same configured /api/v1 URL. Web session cookies stay server-side; Flutter uses secure storage. Flutter assistant -> POST /ai/chat -> [server implementation unknown]. Email/SMS, jobs, storage, payment gateways and AI provider: not found in available source.

These are two neighboring application checkouts, not a verified monorepo. Web package-lock.json and Flutter pubspec.lock are present. No backend repository was found under Documents by searching for nest-cli.json, schema.prisma and ORM configuration names.

3. Existing module inventory

Module	Frontend route	Client API path	Status / evidence note
Authentication	/login, /register, recovery, settings	/auth/*	PARTIAL: Forms, cookie renewal and session controls; real auth QA limited
Organization/onboarding	/settings	/organizations, /invitations/accept	PARTIAL: Create/switch/update UI; onboarding flow and server rules unverified

Module	Frontend route	Client API path	Status / evidence note
Team/roles	/team	members, invitations	PARTIAL: Invite/update/remove forms; server permission rules unverified
Dashboard	/dashboard	/dashboard/summary and related reads	PARTIAL: API-backed UI; historical read QA
Customers/leads	/customers, /customers/[id]	/customers, notes, ledger, relations	PARTIAL: Lead type in customer form; no dedicated pipeline
Products/categories	/products, detail, categories	/products, /product-categories	PARTIAL: CRUD forms and categories; success paths partly historically QA
Inventory	/inventory, detail, low-stock, movements	/inventory/*	PARTIAL: Read, adjustment, stocktake forms; server integrity unverified
Orders/sales	/orders, /orders/[id]	/orders/*, /sales/*	PARTIAL: Draft creation historically QA; transitions not QA
Payments/receivables/refunds	/payments, detail, receivables, refunds	/payments, /receivables, /refunds	PARTIAL: Forms and views; monetary mutation QA absent
Targets	/sales-targets, detail	/targets/*	PARTIAL: Forms and progress read; live mutations unverified
Expenses/categories	/expenses, detail, categories	/expenses/*, /expense-categories	PARTIAL: Forms and read UI; live mutations unverified
Reports/export	/reports, /exports/[kind]	/sales/*, summaries, /exports/*.csv	PARTIAL: Read reports historically QA; CSV proxy exists
Activities/reminders	/activities, detail, reminders	/activities/*, /reminders/*	PARTIAL: Note historically QA; reminders unverified
Notifications	/notifications, settings	/notifications/*, preferences	PARTIAL: Inbox and preference forms; delivery unverified
Subscription	/settings	GET /subscription	PARTIAL: Read-only plan/status display; no billing UI
AI assistant - web	/ai-assistant	none in web	PLACEHOLDER: Explicit unavailable state
AI assistant - mobile	/assistant	POST /ai/chat client call	NOT VERIFIED: Chat UI exists; server capability unknown
Invoice/receipt printing	none	none in inspected clients	NOT FOUND: No print/PDF implementation found
Audit logs/suppliers/purchases/branches/jobs	none	none in available clients	NOT FOUND: Backend absence cannot be asserted

All PARTIAL ratings reflect client implementation and limited QA, not a judgment that the backend is incomplete. Database entities inferred from API fields (customer_id, product_id, organization_id, order_id) are candidate relationships, not verified tables or constraints.

4. Frontend route and UX audit

Discovered 29 page.tsx routes, plus /exports/[kind] and /session/renew route handlers. Auth routes: /login, /register, /forgot-password, /reset-password. Main screens: /dashboard; /customers and detail; /orders and detail; /products, detail and categories; /inventory, detail, low-stock and movements; /payments, detail, receivables and refunds; /sales-targets and detail; /expenses, detail and categories; /reports; /activities, detail and reminders; /team; /settings; /notifications; /ai-assistant. / redirects to /dashboard. Evidence: app/ tree; docs/SCREEN_MAP.md.

Shared ResourcePage supplies no-store list reads, search on selected screens, 20-row pagination, empty/error/403 states and tables. DetailPage fetches related rows; most nested objects are not rendered. OperationForm and lib/operations.ts provide fixed actions and validation. Responsive sidebar and bottom navigation are implemented in components/shell/app-shell.tsx and CSS, but visual behavior was not tested here. The top-bar search button is explicitly disabled. Forms still request raw UUIDs for linked records; line items use JSON-oriented hints/editor. These are significant daily-workflow friction points. Evidence: components/shared/resource-page.tsx, detail-page.tsx, app-shell.tsx, lib/operations.ts.

Historical QA note docs/qa-live-2026-10-19.md reports authenticated checks for login, customer/product creation and detail, stock adjustment, draft order, customer note, dashboard and reports. It explicitly excludes payment, refund, fulfillment, account changes, invitations and destructive actions. This audit did not repeat those checks. Visual design, touch targets, keyboard flow, and mobile overflow remain NOT VERIFIED.

5. Client API inventory and backend audit boundary

The following is an inventory of API requests present in the web client. It is not a verified NestJS controller or endpoint list. All requests use /api/v1 through lib/api.ts; organization-owned requests carry X-Organization-Id. Permission, DTO, database transaction and server error behavior require backend source and live QA.

Area	Web client requests (method and path family)
Account	POST /auth/register, login, refresh, logout, logout-all, password/forgot, password/reset, password/change; GET /auth/me, /auth/sessions; DELETE /auth/sessions/{id}
Organizations/team	GET/POST /organizations; GET/PATCH /organizations/{id}; GET/PATCH/DELETE /organizations/{id}/members/{id}; POST /organizations/{id}/invitations; POST /invitations/accept
Customers	GET/POST /customers; GET/PATCH/DELETE /customers/{id}; POST archive, restore, notes, activities; GET orders, payments, ledger, activities
Products	GET/POST /products; GET/PATCH/DELETE /products/{id}; POST archive, restore; GET/POST/PATCH/DELETE /product-categories
Inventory	GET /inventory, /inventory/{id}, /inventory/low-stock, /inventory/movements, /inventory/valuation; POST /inventory/adjustments, /inventory/stocktakes
Orders	GET/POST /orders; GET/PATCH /orders/{id}; POST confirm, process, fulfill, cancel, return; GET payments, activities
Money	GET/POST /payments; GET /payments/{id}, /payments/summary, /receivables, /refunds; POST /payments/{id}/reverse, /refunds
Targets/expenses	GET/POST /targets; GET/PATCH /targets/{id}; GET /targets/{id}/progress; POST archive; GET/POST /expenses; GET/PATCH /expenses/{id}; POST void; GET/POST /expense-categories
Reports/exports	GET /sales/summary, trends, by-customer, by-product, inactive-customers; GET /expenses/summary; GET /exports/{customers|orders|payments|expenses}.csv
Activities	GET/POST /activities, /reminders; GET/PATCH /activities/{id}; POST complete; PATCH /reminders/{id}
Settings/notifications	GET /subscription, /notification-preferences, /notifications, /notifications/unread-count; PUT /notification-preferences; PATCH /notifications/{id}/read; POST /notifications/read-all

HTTP handling in client: failed fetch maps to synthetic 503; non-2xx reads JSON error.code/message; 401 on /auth/me redirects to renewal; list 403 becomes a role message; operations show API messages and special-case 409 version_conflict. Other 400/404/413/415/422/429/500/502/504 responses have generic handling if returned; no verified server mapping. lib/api.ts:37-80; app/actions/operations.ts:96-102. No actual controllers, services, DTOs, guards, filters or tests can be audited without the backend.

6. Data, security, concurrency and technical risk

VERIFIED SOURCE: client money entry is converted to integer paisa with safe-integer and upper-bound checks; date/UUID/line input is validated; operations marked idempotent attach an Idempotency-Key; PATCH actions often require expected_version. The organization cookie is cross-checked against memberships when getWorkspace runs. Evidence: app/actions/operations.ts; lib/operations.ts; lib/api.ts.

HIGH / NOT VERIFIED: tenant isolation, DB foreign keys, uniqueness, transactions, exact financial storage, ledger balancing, inventory locking, payment reversal immutability, and idempotency storage cannot be established from client code. Verify server-side with backend source, schema and controlled concurrency tests before marketing as production-safe. A malicious direct API caller bypasses client validation, so all rules belong server-side.

MEDIUM / CODE REVIEW FINDING: generic items() scans nested object arrays and success responses are largely schema-less per docs/API_INTEGRATION.md. A response shape change can produce misleading empty states or wrong array selection. Define versioned response DTOs or generated client types after contract QA. MEDIUM: ResourcePage can request page up to 10,000 and all nonpaged views depend on API defaults; audit server limits and indexes. MEDIUM: reports fetch eight endpoints concurrently each page load with no-store; profile actual load before optimizing. LOW: role-based action visibility is not apparent in generic form rendering; verify server rejects unauthorized actions and hide known forbidden affordances where permission metadata exists.

Security positives visible in web client: HTTP-only cookies, production Secure flag, sameSite=lax, server-only API token use, fixed operation catalog, noncached tenant reads and dangerous-action confirmation. CSRF posture for Server Actions, logout/renew route behavior, CORS, upload validation, rate limits, audit logging and secrets management need server/runtime review. No sensitive values are reproduced here.

7. AI assistant and ten-question capability check

Web: /ai-assistant displays an unavailable state. Flutter: AssistantPage posts question text to /ai/chat and displays answer text; it advertises English, Urdu and Roman Urdu in UI. No prompt, provider, model, business tools, permissions, usage records, rate/cost caps, citations or server-side conversation storage are visible. The Flutter message list is in local widget state. Therefore no business question is verified as answered correctly. Evidence: app/ai-assistant/page.tsx; aw_crm/lib/features/ai_assistant/presentation/assistant_page.dart.

Question	Current support assessment
Monthly sales target	Web: NOT SUPPORTED. Mobile: IMPLEMENTED CLIENT REQUEST, server result NOT VERIFIED.
Target achievement	Web: NOT SUPPORTED. Mobile: IMPLEMENTED CLIENT REQUEST, server result NOT VERIFIED.
Show order by number	Web: NOT SUPPORTED. Mobile: IMPLEMENTED CLIENT REQUEST, server result NOT VERIFIED.
Customers with pending payments	Web: NOT SUPPORTED. Mobile: IMPLEMENTED CLIENT REQUEST, server result NOT VERIFIED.
Yesterday sales	Web: NOT SUPPORTED. Mobile: IMPLEMENTED CLIENT REQUEST, server result NOT VERIFIED.
Top five customers	Web: NOT SUPPORTED. Mobile: IMPLEMENTED CLIENT REQUEST, server result NOT VERIFIED.
Low-stock products	Web: NOT SUPPORTED. Mobile: IMPLEMENTED CLIENT REQUEST, server result NOT VERIFIED.
Receivables total	Web: NOT SUPPORTED. Mobile: IMPLEMENTED CLIENT REQUEST, server result NOT VERIFIED.
Who to follow up with	Web: NOT SUPPORTED. Mobile: IMPLEMENTED CLIENT REQUEST, server result NOT VERIFIED.
This month vs last month sales	Web: NOT SUPPORTED. Mobile: IMPLEMENTED CLIENT REQUEST, server result NOT VERIFIED.

Recommended first AI release: read-only, server-authorized tools for dashboard metrics and record lookup; deterministic backend totals; record IDs and date windows in answers; per-organization spend cap; prompt-injection tests; redaction; Urdu/Roman Urdu evaluations. Confirm existing /ai/chat server contract before using it.

8. Printing and invoice/receipt assessment

No browser print, PDF export, A4 invoice, 80mm thermal receipt or payment receipt generation is found in inspected web/Flutter application source. Order numbers appear in UI; invoice numbering and print totals are unverified. An expense receipt_reference field is a reference string, not receipt printing. Recommendation: server-issued immutable invoice/receipt snapshot linked to order/payment; A4 first, payment receipt second, thermal layout third. Include business and customer details, line totals, taxes/discounts, paid/outstanding, reversal status, duplicate copy label and organization branding. Confirm local tax/legal requirements separately before claiming compliance.

9. Missing and incomplete capabilities

Incomplete client workflows: customer/lead pipeline is a customer type rather than a dedicated pipeline; linked-record selectors use IDs; order and payment lifecycle needs QA; ledger rows are generic; reports show small summaries and CSV export; subscription is read-only; notifications are inbox/preferences without verified delivery. Missing in available client UI: supplier and purchasing, formal returns/refund receipt, multi-branch, approval workflow, audit history, import wizard, subscription billing, platform admin, workflow automation, voice and robust AI. Backend existence remains NOT VERIFIED.

Pakistan-oriented opportunities (RECOMMENDATION): WhatsApp-linked follow-up with consent and delivery audit; Urdu/Roman Urdu query quality; cash/bank/easypaisa/jazzcash reconciliation; offline-tolerant mobile sales capture; distributor credit/aging controls; branch stock transfers; PKR paisa precision and timezone-correct reporting. Validate target segment and external integration cost before building.

10. Budget-conscious roadmap

Stage	Work / exit criteria	Cost / dependencies
A - minimal	Obtain backend source and safe QA organization; reconcile API schemas; test financial lifecycle, roles, tenant isolation and conflict paths; replace raw UUIDs; add focused automated tests. Exit: core purchase-to-payment flow passes.	Low incremental infrastructure; engineering effort. Backend access and test fixture required.
B - small budget	Server-issued A4 invoice and payment receipt; ledger/aging UX; audit events and backup restore drill; error monitoring; AI usage cap. Exit: printable, traceable transaction and restore proven.	Low to medium; storage/monitoring cost depends on volume.
C - growth	Purchasing, supplier, multi-branch stock, sales team metrics, selected communications, subscriptions. Exit: pilot customer adoption and reconciled branch transactions.	Medium; depends on reliable core and billing model.
D - mature SaaS	Rules-based automation, predictive insights, permission granularity, enterprise audit and scaling based on measurements. Exit: SLOs and repeatable tenant onboarding.	Medium to high, only with demand and measured load.

11. Top 20 improvements, ranked

#	Improvement	Priority	Value	Complexity	Dependency
1	Acquire backend/schema and map contracts	P0	High	Low	Owner access
2	Verify tenant scoping and permissions	P0	High	Medium	Backend
3	Test payment/refund/reversal invariants	P0	High	Medium	QA tenant
4	Test order-stock concurrency and idempotency	P0	High	Medium	Backend/DB
5	Add financial lifecycle regression tests	P1	High	Medium	Safe fixtures
6	Replace manual UUID selection	P1	High	Medium	List APIs
7	Version success response schemas	P1	High	Medium	Backend contract
8	A4 server-issued invoices	P1	High	Medium	Stable order totals
9	Payment receipts and reprint tracking	P1	High	Medium	Verified ledger
10	Customer ledger/aging workflow	P1	High	Medium	Payment model
11	Unify web/mobile AI contract	P1	Medium	Medium	Backend AI review
12	AI read-only business tools with citations	P2	High	High	Auth, DTOs
13	Role-aware affordances	P2	Medium	Low	Permission metadata
14	Follow-up queue and reminders	P2	High	Medium	Activities
15	CSV import with validation	P2	Medium	Medium	Data rules
16	Monitoring and backup restore checks	P1	High	Medium	Deployment access
17	Supplier/purchase orders	P2	High	High	Inventory integrity
18	Stock transfers and branches	P3	Medium	High	Purchasing
19	Subscription onboarding and billing	P3	High	High	Tenant hardening
20	Urdu/Roman Urdu AI evaluations	P3	Medium	Medium	AI tools

Effort guidance: Low roughly 1-3 developer-days, Medium 4-12, High multiple iterations. These are planning ranges, not estimates validated against backend scope. Additional monthly cost is none/low for most Stage A changes; AI, messaging, storage and monitoring scale with usage and require vendor quotes.

12. First five implementation tasks

1. Contract and source audit

Collect backend repo/schema/OpenAPI; map controllers, guards, DTOs and response schemas. Acceptance: every client request has verified route, auth, permission, validation and response mapping. Risk: unavailable access.

2. Financial integrity QA

Use disposable tenant to test order, payment, refund, reversal, stock and 409 conflicts. Acceptance: ledger/totals reconcile, duplicate requests do not double-post, cross-tenant requests fail. Risk: persistent test data; use isolated fixture.

3. Core regression tests

Add meaningful server/client contract tests for above flows and errors. Acceptance: CI runs without production data or migrations against live DB. Risk: test environment setup.

4. Linked-record selectors

Replace customer/product/category UUID fields and line JSON input with searchable selectors and structured line editor. Acceptance: complete order/payment/stock flows without copying IDs. Risk: pagination and stale choices.

5. Invoice/receipt contract

Define immutable server document data and A4/PDF rendering, then payment receipt. Acceptance: totals match posted records; reprints and reversals are visibly marked. Risk: tax and branding rules need product decision.

13. Maturity assessment

Only source and a historical QA note are available; numerical product scores would imply precision unsupported by this audit. Frontend completeness: PARTIAL, medium confidence. UI/UX quality: NOT ASSESSED visually. Backend completeness, database design, security, performance, production readiness and SaaS scalability: NOT ASSESSED. AI capability: web PLACEHOLDER and mobile CLIENT INTEGRATION ONLY, high confidence on client source. Testing: web automated tests not found; historical QA and Flutter tests exist, medium confidence. These labels are more defensible than 0-10 scores without backend and runtime evidence.

14. Evidence index and audit limitations

aw-crm-web/package.json - verified web dependencies

aw-crm-web/lib/api.ts - server-only API client and organization context

aw-crm-web/lib/operations.ts - fixed mutation catalog and request fields

aw-crm-web/app/actions/operations.ts - validation, versions, idempotency and error mapping

aw-crm-web/components/shared/resource-page.tsx and detail-page.tsx - shared reads and presentation

aw-crm-web/app/reports/page.tsx; app/exports/[kind]/route.ts - reporting and CSV

aw-crm-web/app/ai-assistant/page.tsx - unavailable web AI state

aw-crm-web/docs/qa-live-2026-10-19.md - historical QA claims and exclusions

aw_crm/pubspec.yaml; lib/app/app.dart; lib/features/ai_assistant/presentation/assistant_page.dart - Flutter scope and AI client

Limitations: no backend repository, PostgreSQL schema, production access, authenticated tests, browser visual checks or live API interrogation were used for this report. Backend endpoints above are client-request paths. Database entities, constraints, provider, costs and actual server behavior must be verified before implementation decisions.

15. Instructions for Claude AI

Claude, act as a Principal SaaS Product Architect, AI Solutions Architect, Senior Next.js and NestJS Engineer, and CRM Business Strategist.

Review this complete AW CRM project audit and identify how to transform the existing application into a competitive, AI-first CRM platform for Pakistani businesses.

1. Improvements to existing modules.

2. Missing essential modules.

3. Innovative AI-powered features.

4. UI/UX and workflow improvements.

5. Security, performance, and scalability recommendations.

6. Advanced business automation opportunities.

7. Features that could differentiate AW CRM from competitors.

8. A budget-conscious implementation roadmap.

9. Monetization and subscription opportunities.

10. The 20 highest-value improvements ranked by impact, cost, and complexity.

For each recommendation, explain its business value, user workflow, technical requirements, implementation dependencies, and approximate development effort.

Prioritize practical improvements that build on the existing codebase rather than recommending an unnecessary rewrite.